

分子动力学文件语法高亮 VScode 插件
0.0.7
使用说明

2026 年 3 月 25 日

目 录

一、 软件基本信息	1
1.1 软件名称和版本信息	1
1.2 软件用途	1
1.3 适用对象	1
1.4 主要功能	2
1.5 支持范围	2
1.6 实现特点	2
1.7 文档编写目的	3
二、 安装和配置	3
2.1 系统要求	3
2.2 软件特性	3
2.2.1 颜色编码方式	4
2.2.2 主题兼容性	4
2.3 安装方法	5
2.3.1 从 Visual Studio Marketplace 安装	5
2.3.2 从 VSIX 文件安装	5
2.3.3 从命令行安装	5
2.4 验证安装	6
2.5 配置选项	6
2.5.1 主题适配	6
2.5.2 文件关联	6
2.6 更新和维护	6
2.6.1 更新方式	6
2.6.2 版本历史	7
2.6.3 问题反馈	7
2.7 卸载	7
三、 使用示例	7
3.1 NAMD/CHARMM 文件格式	7
3.1.1 蛋白质数据库文件 (PDB)	7
3.1.2 残基拓扑文件 (RTF)	8
3.1.3 蛋白质结构文件 (PSF)	9
3.1.4 CHARMM 参数文件 (PRM)	9

3.2	Amber/AmberTools 文件格式	10
3.2.1	Amber 拓扑文件 (PRMTOP)	10
3.2.2	Amber 库文件 (LIB)	12
3.2.3	Amber 预处理输入文件 (PREPIN)	13
3.2.4	Amber 力场修改文件 (FRCMOD)	13
3.2.5	AmberTools 输入脚本 (TLEAP/相关.in 文件)	14
3.2.6	Antechamber 主链文件 (MC)	14
3.2.7	Antechamber 文件 (AC)	16
3.3	Gromacs 文件格式	17
3.3.1	Gromacs 坐标文件 (GRO)	17
3.3.2	Gromacs 原子类型文件 (ATP)	17
3.3.3	Gromacs 原子重命名文件 (ARN)	18
3.3.4	Gromacs 残基类型文件 (RTP)	18
3.3.5	Gromacs 氢数据库文件 (HDB)	18
3.3.6	Gromacs 残基到珠映射文件 (R2B)	19
3.3.7	Gromacs 终端数据库文件 (TDB)	19
3.3.8	Gromacs 突变拓扑文件 (MTP)	19
3.3.9	Gromacs 虚拟位点定义文件 (VSD)	20
3.4	小分子文件格式	20
3.4.1	MOL2 分子结构文件 (MOL2)	20
3.4.2	结构数据文件 (SDF)	20
3.5	增强采样文件格式	20
3.5.1	PLUMED 输入文件	20
3.6	实际用途	21
3.6.1	文本阅读和人工检查	21
3.6.2	跨格式编辑场景	22
四、	未来发展	22
4.1	计划改进	22
4.2	总结	22

一、 软件基本信息

1.1 软件名称和版本信息

表 1 md-highlighter 软件基本信息

项目	描述
软件全称	分子动力学文件语法高亮 VSCode 插件
软件简称	md-highlighter
版本号	0.0.7
软件分类	应用软件
软件类型	VSCode 扩展插件
开发语言	JSON 配置文件
运行环境	VSCode1.70.0 及以上版本
联系方式	gxf1212@zju.edu.cn
仓库地址	https://github.com/gxf1212/md-highlighter
程序规模	配置文件以及 23 个语法定义文件，共 1761 行
当前公开版本	0.0.7

1.2 软件用途

md-highlighter 用于在 VSCode 中为分子动力学相关文本文件提供语法高亮。本项目不涉及计算、建模或结果分析，主要在编辑器中完成文件类型识别、语法元素匹配和颜色作用域分配，用于支持专业输入文件的阅读和修改。

从仓库实现看，其工作流程较为明确：先在package.json中注册语言标识、文件扩展名和语法文件路径，再由language-configuration.json提供通用的括号、注释和自动补全配置；各类具体格式的匹配规则写在syntaxes/*.tmLanguage.json中，最后由 VSCode 根据匹配结果为文本片段赋予作用域，再由当前主题显示颜色。

1.3 适用对象

本工具面向需要直接编辑分子模拟输入文件的用户。比较常见的使用者，一类是使用 NAMD、CHARMM、Amber、AmberTools、Gromacs、PLUMED 的研究人员；一类是在 VSCode 中查看力场、拓扑、坐标和参数文件的实验人员；另外也包括需要补充或维护这类文本语法高亮规则的开发者。

1.4 主要功能

当前版本与源码能够直接对应的功能主要包括以下几项。第一是文件识别，项目注册了 23 个语言 ID，并将常见扩展名或文件名模式映射到对应语言。第二是语法高亮，不同文件格式分别定义匹配规则，对关键字、原子名、残基名、编号、坐标和参数值等内容着色。第三是作用域映射，尽量复用 VSCode 主题可识别的标准 TextMate 作用域，例如 `entity.name`、`constant.numeric`、`support.type`。此外，该扩展属于轻量级编辑器扩展，不包含后台服务、数据库或独立执行程序，安装后由编辑器直接加载。

1.5 支持范围

按照 `package.json` 当前注册结果，本软件共注册 23 个语言 ID，对应 23 个语法定义文件，覆盖以下文件类型：

表 2 已注册的语言和文件类型

类别	语言 ID 或扩展名
NAMD 或 CHARMM	<code>rtf</code> (<code>.rtf</code> 、 <code>.str</code> 、 <code>.inp</code> 、 <code>.prm</code> 、 <code>.par</code>)、 <code>pdb</code> (<code>.pdb</code>)、 <code>psf</code> (<code>.psf</code>)
Amber 或 Amber-Tools	<code>in</code> (<code>.in</code> , 并匹配 <code>leaprc.*</code>)、 <code>prmtop</code> (<code>.prmtop</code> 、 <code>.parm7</code>)、 <code>inpcrd</code> (<code>.inpcrd</code> 、 <code>.rst7</code>)、 <code>prepin</code> (<code>.prepin</code> 、 <code>.prepi</code>)、 <code>frcmod</code> (<code>.frcmod</code> 及若干参数文件名模式)、 <code>lib</code> (<code>.lib</code> 、 <code>.off</code>)、 <code>ac</code> (<code>.ac</code>)、 <code>mc</code> (<code>.mc</code>)
Gromacs 小分子	<code>gro</code> 、 <code>atp</code> 、 <code>arn</code> 、 <code>rtp</code> 、 <code>hdb</code> 、 <code>r2b</code> 、 <code>tdb</code> 、 <code>mtp</code> 、 <code>vsd</code> <code>mol2</code> 、 <code>sdf</code> (<code>.sdf</code> 、 <code>.mol</code>)
其他	<code>plumed</code> (<code>.plumed.dat</code>)

1.6 实现特点

本软件的实现特点可以直接从代码文件中对应出来。当前的组织方式基本是一种语言对应一个 `.tmLanguage.json` 文件，后续维护时可按文件分别调整。各语法文件内部主要使用内联正则表达式描述文本结构，没有额外的编译步骤。高亮范围基于文本模式匹配实现，不依赖外部命令，也不读取网络数据。概括而言，当前版本主要解决阅读和编辑过程中的字段区分问题，不负责语义计算、参数求解或自动修复。

1.7 文档编写目的

本说明文档用于说明 **md-highlighter** 的实际功能、安装方式、支持文件和使用边界，供软件著作权登记时与申请表、源程序文档进行对应核对。文中描述以当前仓库代码和随附截图为依据，能写到哪一步就写到哪一步，不扩展到程序里还没有实现的功能。

二、 安装和配置

2.1 系统要求

md-highlighter 是一个 VSCode 扩展。这个项目本身没有单独的后端程序，也不需要数据库。实际使用时，主要要求还是 VSCode 能正常加载扩展配置和 TextMate 语法文件。

表 3 md-highlighter 系统要求

要求项目	规格
操作系统	Windows、macOS、Linux，和 VSCode 支持范围一致
VSCode 版本	1.70.0 及以上版本，对应 <code>package.json</code> 中的 <code>engines.vscode</code> 配置
硬件要求	无单独硬件要求，通常与 VSCode 编辑普通文本文件时的要求一致
运行依赖	无额外服务端依赖，主要由 VSCode 加载本扩展中的 JSON 配置文件
网络要求	安装扩展时可能需要网络；本地使用语法高亮时不依赖网络连接

2.2 软件特性

本章仅说明当前仓库中已经实现的内容，不额外扩展。**md-highlighter** 的主要功能是为分子模拟相关文本文件提供语法高亮。当前版本在 `package.json` 中注册了 23 个语言 ID，对应 23 个语法定义文件，范围包括 NAMD/CHARMM、Amber/AmberTools、Gromacs、小分子结构文件以及 PLUMED 输入文件。

2.2.1 颜色编码方式

这个扩展本身并不直接写死每一种颜色，而是给文本打上 **TextMate** 作用域，最后由当前主题决定显示效果。结合现有语法文件，当前版本里常见的作用域用法大致如下：

- **名称类字段**：原子名、残基名、分段名称等，一般使用 `entity.name.*` 或相近作用域
- **数值类字段**：坐标、电荷、质量、一般参数值，常见为 `constant.numeric.*`
- **类型类字段**：原子类型、参数类型、格式类型等，常见为 `support.type.*`
- **注释和记录头**：注释行、说明行、记录关键字，会根据具体文件使用 `comment.*`、`keyword.*` 等作用域

这种实现方式下，语法文件主要负责“分段”和“标记”，无需在扩展中另外维护颜色表。

2.2.2 主题兼容性

README 中写到当前项目在 **Atom One Light** 主题下做过显示检查。至于其他主题，本扩展仍按 **VSCode** 的作用域取色机制显示；但不同主题之间的颜色差异较大，实际显示效果应以用户本地主题为准，因此本文档不将其表述为统一结论。

如果需要手动调某一类字段的颜色，可以在 `settings.json` 中覆盖指定作用域。例如：

```
{
  "editor.tokenColorCustomizations": {
    "textMateRules": [
      {
        "scope": "entity.name.tag.atom-name",
        "settings": {
          "foreground": "#0066CC",
          "fontStyle": "bold"
        }
      }
    ]
  }
}
```

2.3 安装方法

2.3.1 从 Visual Studio Marketplace 安装

如果该版本已经发布到扩展市场，可以直接在 VSCode 里搜索安装。操作步骤如下：

- 打开 VSCode
- 进入扩展视图，或者按 `Ctrl+Shift+X`
- 搜索 `md-highlighter`
- 选择发布者为 `gxf1212` 的扩展并安装

2.3.2 从 VSIX 文件安装

如果是从仓库发布页拿到 VSIX 包，也可以手动安装：

- 打开 VSCode
- 按 `Ctrl+Shift+P` 打开命令面板
- 输入 `Extensions: Install from VSIX...`
- 选择对应的 VSIX 文件
- 等待 VSCode 完成安装

2.3.3 从命令行安装

如果本机已经能使用 `code` 命令，也可以直接执行：

```
code --install-extension gxf1212.md-highlighter
```


2.4 验证安装

安装完成后，可以打开一个已经注册的文件类型，查看语言模式和高亮是否生效。当前版本可按下面的方法确认：

- 打开一个 `.pdb`、`.rtf`、`.gro` 或 `.plumed.dat` 文件
- 查看 VSCode 右下角的语言模式是否切换到本扩展注册的语言 ID
- 查看文件内容是否已经按字段出现分段高亮，而不是整段普通文本
- 在扩展列表中确认 `md-highlighter` 处于已启用状态

2.5 配置选项

当前版本没有自定义设置项，安装后即可使用。和显示有关的调整主要还是走 VSCode 自身的主题和文件关联能力，而不是本扩展单独提供一套配置面板。

2.5.1 主题适配

主题适配主要取决于当前主题对 `TextMate` 作用域的解释方式。当前材料中只保留能够确认的信息：项目作者在 `Atom One Light` 主题下查看过显示效果。若用户本地使用其他主题，通常也能看到高亮，但具体颜色深浅和对比度仍取决于对应主题本身的配色方案。

2.5.2 文件关联

扩展在 `package.json` 里注册了扩展名和部分文件名模式。大多数情况下，打开文件时会自动进入对应语言模式。比如：

- `.prmtop` 和 `.parm7` 会进入 `prmtop` 模式
- `.prepin` 和 `.prepi` 会进入 `prepin` 模式
- `leaprc.test` 会按 `in` 语言模式处理
- `.plumed.dat` 会进入 `plumed` 模式

如果文件后缀不标准，或者 VSCode 没自动识别，也可以手动切换语言模式，再选择本扩展注册的条目。

2.6 更新和维护

2.6.1 更新方式

如果扩展通过市场安装，更新方式和普通 VSCode 扩展一致，通常由编辑器自动处理。如果使用的是本地 `VSIX` 包，则需要在版本更新后重新安装新版包。

2.6.2 版本历史

CHANGELOG.md 记录了 0.0.1 到 0.0.7 的变更。当前版本 0.0.7 的明确增量是增加了 vsd 文件支持。更早几个版本则陆续补充了 parm7、rst7、plumed、prepin、frcmod、atp、rtp、hdb、r2b、tdb、arn 等格式。

2.6.3 问题反馈

当前仓库里保留了两个反馈渠道：

- GitHub Issues: <https://github.com/gxf1212/md-highlighter/issues>
- 邮件: <mailto:gxf1212@zju.edu.cn>

2.7 卸载

卸载方式和普通 VSCode 扩展一样：

- 打开扩展管理界面
- 找到 md-highlighter
- 点击卸载
- 如有需要，重启 VSCode

卸载后只是移除语言注册和语法高亮配置，不会改动用户原来的文本文件。

三、 使用示例

md-highlighter 安装后会在 VSCode 中按文件类型自动加载相应语法文件。下面各节按仓库中已经存在的语法定义说明高亮范围，不另外引入未实现的功能。软件实现的是文本匹配和作用域标注，不做分子结构计算，也不做力场参数求解。

3.1 NAMD/CHARMM 文件格式

3.1.1 蛋白质数据库文件（PDB）

PDB 文件的高亮规则定义在 `syntaxes/pdb.tmLanguage.json`。相关规则先将 CRYST、REMARK、HEADER、TITLE、COMPND、SOURCE、JRNL、CONECT 等记录作为说明性记录处理；再单独匹配 CRYST1、TER、END、HELIX、SHEET、SSBOND；对 ATOM 和 HETATM 行，则按固定列宽继续拆出原子序号、原子名、残基名、链标识、残基编号、坐标、占有率、温度因子、段名、电荷等字段。如图图 1 所示，PDB 文件中的记录头和原子数据列在编辑器中已有分层显示。

1	REMARK	GENERATE	LIGAND							
2	REMARK	DATE:	12/13/22	6: 1:35	CREATED BY	USER:	apache			
3	ATOM	1	P1	LIG	L	1	5.998	-7.223	-5.561	1.00 0.00 LIG
4	ATOM	2	P2	LIG	L	1	8.461	-5.541	-5.455	1.00 0.00 LIG
5	ATOM	3	P3	LIG	L	1	10.717	-7.460	-5.915	1.00 0.00 LIG
6	ATOM	4	C1	LIG	L	1	2.820	-2.950	-3.870	1.00 0.00 LIG
7	ATOM	5	N1	LIG	L	1	0.514	-3.676	0.105	1.00 0.00 LIG
8	ATOM	6	O1	LIG	L	1	4.280	-0.687	-6.398	1.00 0.00 LIG
9	ATOM	7	C2	LIG	L	1	2.473	-4.424	-2.235	1.00 0.00 LIG
10	ATOM	8	N2	LIG	L	1	1.831	-2.376	-3.112	1.00 0.00 LIG
11	ATOM	9	O2	LIG	L	1	5.075	-5.881	-5.597	1.00 0.00 LIG
12	ATOM	10	C3	LIG	L	1	4.979	-5.230	-6.883	1.00 0.00 LIG
13	ATOM	11	N3	LIG	L	1	0.029	-1.771	-1.059	1.00 0.00 LIG
14	ATOM	12	O3	LIG	L	1	3.162	-3.808	-6.144	1.00 0.00 LIG
15	ATOM	13	C4	LIG	L	1	4.467	-3.792	-6.742	1.00 0.00 LIG
16	ATOM	14	N4	LIG	L	1	1.174	-1.206	-3.100	1.00 0.00 LIG
17	ATOM	15	O4	LIG	L	1	6.460	-2.319	-6.562	1.00 0.00 LIG
18	ATOM	16	C5	LIG	L	1	5.315	-2.847	-5.874	1.00 0.00 LIG
19	ATOM	17	N5	LIG	L	1	0.799	-1.416	-6.441	1.00 0.00 LIG
20	ATOM	18	O5	LIG	L	1	5.971	-7.694	-4.129	1.00 0.00 LIG
21	ATOM	19	C6	LIG	L	1	4.295	-1.764	-5.442	1.00 0.00 LIG
22	ATOM	20	O6	LIG	L	1	8.193	-4.170	-6.056	1.00 0.00 LIG
23	ATOM	21	C7	LIG	L	1	2.956	-2.598	-5.384	1.00 0.00 LIG
24	ATOM	22	O7	LIG	L	1	11.986	-6.764	-5.468	1.00 0.00 LIG
25	ATOM	23	C8	LIG	L	1	0.259	-0.967	-2.129	1.00 0.00 LIG
26	ATOM	24	O8	LIG	L	1	5.400	-8.109	-6.634	1.00 0.00 LIG
27	ATOM	25	C9	LIG	L	1	1.602	-3.288	-2.013	1.00 0.00 LIG
28	ATOM	26	O9	LIG	L	1	8.491	-5.563	-3.947	1.00 0.00 LIG
29	ATOM	27	C10	LIG	L	1	0.708	-2.937	-0.985	1.00 0.00 LIG
30	ATOM	28	O10	LIG	L	1	10.858	-8.155	-7.259	1.00 0.00 LIG
31	ATOM	29	C11	LIG	L	1	3.167	-4.190	-3.397	1.00 0.00 LIG
32	ATOM	30	O11	LIG	L	1	7.354	-6.536	-6.069	1.00 0.00 LIG
33	ATOM	31	C12	LIG	L	1	1.775	-1.910	-5.951	1.00 0.00 LIG
34	ATOM	32	O12	LIG	L	1	9.724	-6.179	-6.216	1.00 0.00 LIG
35	ATOM	33	O13	LIG	L	1	10.090	-8.337	-4.854	1.00 0.00 LIG
36	ATOM	34	H1	LIG	L	1	5.140	-0.761	-6.846	1.00 0.00 LIG
37	ATOM	35	H2	LIG	L	1	2.575	-5.311	-1.624	1.00 0.00 LIG

图 1 PDB 文件语法高亮演示 (.pdb 格式)

从实际显示效果看，记录名与数据行分别处理；原子记录内部又按固定列位置区分原子序号、原子名、残基名、链 ID、三列坐标和元素符号等字段。该规则可用于在编辑器中查看文本布局，并核对坐标列、残基编号和记录类型。

3.1.2 残基拓扑文件 (RTF)

RTF 和部分 CHARMM 参数文件共用 `syntaxes/rtf.tmLanguage.json`。这份语法文件里既有拓扑项，也兼顾了参数项。实际匹配内容包括以 ! 和 * 开头的注释行，MASS、RESI、PRES、ATOM、BOND、ANGLE、DIHEDRAL、IMPR、PATCH、LONEPAIR 等关键字，以及这些关键字后面的质量编号、元素符号、残基名、电荷数值、原子名称和原子类型。如图 2 所示，注释、关键字和参数列之间已经区分开。

在该文件里，注释、拓扑关键字、残基名、电荷、原子名和参数区数据是分开显示的；MASS 行以及 RESI/PRES 行还有单独规则。这类高亮可用于查看残基定义、键接关系以及力场项的文本组织方式，并定位拓扑段落。

```
1  * Topologies generated by
2  * CHARMM General Force Field (CGenFF) program version 2.5.1
3  *
4  36 1
5
6  ! "penalty" is the highest penalty score of the associated parameters.
7  ! Penalties lower than 10 indicate the analogy is fair; penalties between 10
8  ! and 50 mean some basic validation is recommended; penalties higher than
9  ! 50 indicate poor analogy and mandate extensive validation/optimization.
10
11  RESI lig          -4.000 ! param penalty= 288.500 ; charge penalty= 87.046
12  GROUP           ! CHARGE  CH_PENALTY
13  ATOM P1          PG1    1.505 !    0.000
14  ATOM P2          PG1    1.505 !    0.000
15  ATOM P3          PG2    1.105 !    0.000
16  ATOM C1          CG2R57 0.299 !   60.060
17  ATOM N1          NG2S3  -0.764 !   22.006
18  ATOM O1          OG311  -0.647 !   15.381
19  ATOM C2          CG2R51 -0.226 !   20.406
20  ATOM N2          NG2RC0 -0.073 !   69.844
21  ATOM O2          OG303  -0.621 !    0.000
22  ATOM C3          CG321  -0.081 !    0.304
23  ATOM N3          NG2R62 -0.781 !   28.183
24  ATOM O3          OG3C51 -0.552 !   35.011
25  ATOM C4          CG3C51 0.143 !   21.734
26  ATOM N4          NG2R62 -0.475 !   45.406
27  ATOM O4          OG311  -0.647 !    0.040
28  ATOM C5          CG3C51 0.106 !    7.152
29  ATOM N5          NG1T1  -0.460 !    4.472
30  ATOM O5          OG2P1  -0.816 !    0.000
31  ATOM C6          CG3C51 0.106 !   24.250
32  ATOM O6          OG2P1  -0.835 !    0.000
33  ATOM C7          CG3C50 0.436 !   87.046
34  ATOM O7          OG2P1  -0.900 !    0.000
35  ATOM C8          CG2R64 0.795 !   28.027
36  ATOM O8          OG2P1  -0.816 !    0.000
37  ATOM C9          CG2RC0 -0.145 !   53.697
38  ATOM O9          OG2P1  -0.835 !    0.000
39  ATOM C10         CG2R64 0.496 !   38.049
40  ATOM O10         OG2P1  -0.900 !    0.000
41  ATOM C11         CG2R51 -0.297 !   23.336
```

图 2 RTF 文件语法高亮演示 (.rtf 格式)

3.1.3 蛋白质结构文件 (PSF)

PSF 文件的高亮规则定义在 `syntaxes/psf.tmLanguage.json`。该规则重点区分 PSF 文件头、说明性记录、各个分段标题、总数值以及原子数据行中的字段。对于长文件而言，分段标题和数据主体可以分别显示。如图图 3 所示，PSF 文件中标题区、分段标识和正文数据区能够分开显示。

其实现方式较为明确，文件头、分段标识、计数字段和原子相关数据采用不同作用域。对较长的 PSF 文本而言，可以先区分分段标题和数据区域。

3.1.4 CHARMM 参数文件 (PRM)

扩展把 .prm 文件也归入 rtf 语言配置，因此它的语法高亮仍然来自 `syntaxes/rtf.tmLanguage.json`。在该分析项中，主要是对 BONDS、ANGLES、DIHEDRALS

```
28 REMARKS patch NTER PR3:2
29 REMARKS patch CTER PR4:191
30 REMARKS patch ACE PR4:56
31
32 19110 !NATOM
33 1 MG 810 MG MG MG 2.000000 24.3050 0
34 2 MG 811 MG MG MG 2.000000 24.3050 0
35 3 ZN 812 ZN2 ZN ZN 2.000000 65.3700 0
36 4 ZN 813 ZN2 ZN ZN 2.000000 65.3700 0
37 5 WAT 105 TIP3 OH2 OT -0.834000 15.9994 0
38 6 WAT 105 TIP3 H1 HT 0.417000 1.0080 0
39 7 WAT 105 TIP3 H2 HT 0.417000 1.0080 0
40 8 WAT 197 TIP3 OH2 OT -0.834000 15.9994 0
41 9 WAT 197 TIP3 H1 HT 0.417000 1.0080 0
42 10 WAT 197 TIP3 H2 HT 0.417000 1.0080 0
43 11 WAT 432 TIP3 OH2 OT -0.834000 15.9994 0
44 12 WAT 432 TIP3 H1 HT 0.417000 1.0080 0
45 13 WAT 432 TIP3 H2 HT 0.417000 1.0080 0
46 14 WAT 508 TIP3 OH2 OT -0.834000 15.9994 0
47 15 WAT 508 TIP3 H1 HT 0.417000 1.0080 0
48 16 WAT 508 TIP3 H2 HT 0.417000 1.0080 0
49 17 WAT 610 TIP3 OH2 OT -0.834000 15.9994 0
50 18 WAT 610 TIP3 H1 HT 0.417000 1.0080 0
51 19 WAT 610 TIP3 H2 HT 0.417000 1.0080 0
52 20 WAT 644 TIP3 OH2 OT -0.834000 15.9994 0
53 21 WAT 644 TIP3 H1 HT 0.417000 1.0080 0
54 22 WAT 644 TIP3 H2 HT 0.417000 1.0080 0
55 23 WAT 803 TIP3 OH2 OT -0.834000 15.9994 0
56 24 WAT 803 TIP3 H1 HT 0.417000 1.0080 0
57 25 WAT 803 TIP3 H2 HT 0.417000 1.0080 0
58 26 WAT 873 TIP3 OH2 OT -0.834000 15.9994 0
59 27 WAT 873 TIP3 H1 HT 0.417000 1.0080 0
60 28 WAT 873 TIP3 H2 HT 0.417000 1.0080 0
```

图 3 PSF 文件语法高亮演示（.psf 格式）

、IMPROPERS、CMAP、NONBONDED、NBFIX 等参数区名称做匹配，再把原子类型、浮点数参数和部分节标题分开显示。如图 4 所示，参数区标题与数值列在当前版本里已经能够形成层次。

PRM 部分主要区分参数区标题、原子类型字符串和数值参数。其作用主要体现在阅读和修改参数文本时，对节标题、类型名和数值列进行分层显示。

3.2 Amber/AmberTools 文件格式

3.2.1 Amber 拓扑文件（PRMTOP）

PRMTOP 文件使用 `syntaxes/prmtop.tmLanguage.json`。这份规则是按块写的。先识别 `%VERSION`、`%FLAG`、`%FORMAT` 三类标记，再对 `ATOM_NAME`、`ATOMIC_NUMBER`、`MASS`、`RESIDUE_LABEL`、`AMBER_ATOM_TYPE` 等数据块分别处理。实际输出为：格式字符串、原子名、原子序数、残基名、原子类型和数值项会落在不同作用域里。如图 5 所示，PRMTOP 文件在编辑器中更接近按块阅读的效果。

实际显示时，`%FLAG/%FORMAT` 标记、格式说明、原子名块、残基标签块和

```

BONDS
CG1N1  CG3C50  400.00      1.4700 ! Lig , from CG1N1 CG321,
CG2R57  CG3C50  350.00      1.5100 ! Lig , from CG2R51 CG3C51
CG2R57  NG2RC0  400.00      1.3710 ! Lig , from CG2R51 NG2RC0
CG3C50  CG3C51  195.00      1.5180 ! Lig , from CG3C50 CG3C51
CG3C50  OG3C51  350.00      1.4250 ! Lig , from CG3C51 OG3C51
NG2R62  NG2RC0  420.00      1.3200 ! Lig , from NG2R62 NG2R62

ANGLES
CG3C50  CG1N1  NG1T1      21.20      180.00 ! Lig , from CG321
CG2R51  CG2R57  CG3C50     115.00      109.00 ! Lig , from CG2R51
CG2R51  CG2R57  NG2RC0     130.00      108.20 ! Lig , from CG2R51
CG3C50  CG2R57  NG2RC0     170.00      112.00 ! Lig , from CG3C51
...

DIHEDRALS
CG2R57  CG2R51  CG2R51  CG2RC0      2.0000  2  180.00 ! Lig ,
CG2R57  CG2R51  CG2R51  HGR51       1.5000  2  180.00 ! Lig ,
CG2R51  CG2R51  CG2R57  CG3C50      4.0000  2  180.00 ! Lig ,
CG2R51  CG2R51  CG2R57  NG2RC0     16.0000  2  180.00 ! Lig ,
HGR51   CG2R51  CG2R57  CG3C50      2.9000  2  180.00 ! Lig ,
HGR51   CG2R51  CG2R57  NG2RC0      3.7000  2  180.00 ! Lig ,
CG2R51  CG2R51  CG2RC0  CG2R64      3.0000  2  180.00 ! Lig ,
HGR51   CG2R51  CG2RC0  CG2R64      2.8000  2  180.00 ! Lig ,
CG2R51  CG2R57  CG3C50  CG1N1       3.5914  2  180.00 ! Lig ,
CG2R51  CG2R57  CG3C50  CG3C51       0.3500  3  180.00 ! Lig ,
CG2R51  CG2R57  CG3C50  OG3C51       0.2800  1  180.00 ! Lig ,
CG2R51  CG2R57  CG3C50  OG3C51       0.9800  2  180.00 ! Lig ,
CG2R51  CG2R57  CG3C50  OG3C51       1.7500  3  180.00 ! Lig ,
...

IMPROPERS

END

```

图 4 CHARMM PRM 参数文件语法高亮演示（.prm 格式）

数值块会被分层处理。对 PRMTOP 这类按块组织的数据文件来说，可以据此定位常见分段。

.parm7 扩展名在 package.json 中与 prmtop 语言 ID 共用同一套语法定义，因此它的高亮方式和 PRMTOP 相同。仓库中没有为 .parm7 另外再写一套规则，而是直接复用 syntaxes/prmtop.tmLanguage.json 中的分块匹配逻辑。如图 6 所示，PARM7 文件的显示层次与 PRMTOP 基本一致。

因为直接复用了 PRMTOP 的规则，所以这里看到的也是同一套标记、字段

```
1 %VERSION VERSION_STAMP = V0001.000 DATE = 03/25/23 23:14:47
2 %FLAG TITLE
3 %FORMAT(20a4)
4 default_name
5 %FLAG POINTERS
6 %FORMAT(10I8)
7 64715 18 62057 2598 5752 3511 11559 11124 0 0
8 107514 20278 2598 3511 11124 67 152 192 38 0
9 0 0 0 0 0 0 0 1 24 0
10 0
11 %FLAG ATOM_NAME
12 %FORMAT(20a4)
13 N H1 H2 H3 CA HA CB HB CG1 HG11HG12HG13CG2 HG21HG22HG23C O N H
14 CA HA CB HB2 HB3 CG OD1 OD2 C O N CD HD2 HD3 CG HG2 HG3 CB HB2 HB3
15 CA HA C O N H CA HA CB HB2 HB3 CG HG2 HG3 CD OE1 OE2 C O N
16 H CA HA CB HB CG2 HG21HG22HG23CG1 HG12HG13CD1 HD11HD12HD13C O N H
17 CA HA CB HB2 HB3 CG HG2 HG3 CD OE1 OE2 C O N H CA HA CB HB2 HB3
18 CG HG2 HG3 CD OE1 OE2 C O N H CA HA CB HB2 HB3 OG HG C O N
```

图 5 Amber PRMTOP 文件语法高亮演示（.prmtop 格式）

```
1 %VERSION VERSION_STAMP = V0001.000 DATE = 09/26/23 11:05:20
2 %FLAG CTITLE
3 %FORMAT(20a4)
4
5 %FLAG POINTERS
6 %FORMAT(10I8)
7 12834 18 10338 2448 11590 2832 14448 7776 0 0
8 44440 2182 2448 2832 7776 23 42 71 0 0
9 0 0 0 0 0 0 0 1 134 0
10 0
11 %FLAG FORCE_FIELD_TYPE
12 %FORMAT(i2,a78)
13 1 CHARMM36 ALL-ATOM FORCE FIELD
14 %FLAG ATOM_NAME
15 %FORMAT(20a4)
16 N C12 H12AH12BC13 H13AH13BH13CC14 H14AH14BH14CC15 H15AH15BH15CC11 H11AH11BP
17 O13 O14 O12 O11 C1 HA HB C2 HS O21 C21 O22 C22 H2R H2S C3 HX HY O31 C31
18 O32 C32 H2X H2Y C23 H3R H3S C24 H4R H4S C25 H5R H5S C26 H6R H6S C27 H7R H7S C28
19 H8R H8S C29 H91 C210H101C211H11RH11SC212H12RH12SC213H13RH13SC214H14RH14SC215H15R
20 H15SC216H16RH16SC217H17RH17SC218H18RH18SH18TC33 H3X H3Y C34 H4X H4Y C35 H5X H5Y
```

图 6 Amber PARM7 文件语法高亮演示（.parm7 格式）

块和数值高亮方式。对应的文件范围就是使用 .parm7 扩展名的 Amber 拓扑文件。

3.2.2 Amber 库文件（LIB）

LIB/OFF 文件的语法定义位于 `syntaxes/lib.tmLanguage.json`。相关规则基本按照文件本身的组织方式编写，主要识别 `!entry` 记录、残基名、unit 类型、原子名、残基编号、原子编号、元素编号、电荷以及纯数字索引行。以 `!entry` 开头的结构段单独处理，因此头部标识和后续数据行不会处于同一层级。如图 7 所示，LIB 文件里的条目头和原子数据区是可以分辨开的。

`!entry` 关键字、残基名、原子名、编号和电荷字段会分别显示出来。查看 tLeap 库文件时，可按条目结构逐段阅读，并定位原子列表。


```
1  !!index array str
23 "PHE"
24 "PRO"
25 "SER"
26 "THR"
27 "TRP"
28 "TVR"
29 "VAL"
30 !entry.ALA.unit.atoms table str name str type int typex int resx int flags int
    seq int elmnt dbl chg
31 "N" "N" 0 1 131072 1 7 -0.415700
32 "H" "H" 0 1 131072 2 1 0.271900
33 "CA" "CX" 0 1 131072 3 6 0.033700
34 "HA" "H1" 0 1 131072 4 1 0.082300
35 "CB" "CT" 0 1 131072 5 6 -0.182500
36 "HB1" "HC" 0 1 131072 6 1 0.060300
37 "HB2" "HC" 0 1 131072 7 1 0.060300
38 "HB3" "HC" 0 1 131072 8 1 0.060300
39 "C" "C" 0 1 131072 9 6 0.597300
40 "O" "O" 0 1 131072 10 8 -0.567900
41 !entry.ALA.unit.atomsptinfo table str pname str ptype int ptypex int peLmnt dbl
    pchg
42 "N" "N" 0 -1 0.0
43 "H" "H" 0 -1 0.0
44 "CA" "CX" 0 -1 0.0
45 "HA" "H1" 0 -1 0.0
46 "CB" "CT" 0 -1 0.0
47 "HB1" "HC" 0 -1 0.0
48 "HB2" "HC" 0 -1 0.0
49 "HB3" "HC" 0 -1 0.0
50 "C" "C" 0 -1 0.0
51 "O" "O" 0 -1 0.0
52 !entry.ALA.unit.boundingBox array dbl
53 -1.000000
54 0.0
55 0.0
56 0.0
57 0.0
58 !entry.ALA.unit.childsequence single int
59 2
```

图 7 Amber LIB 文件语法高亮演示 (.lib 格式)

3.2.3 Amber 预处理输入文件 (PREPIN)

PREPIN 文件的规则定义在 `syntaxes/prepin.tmLanguage.json`。除原子表外，该规则还包含若干控制项，能够识别 `remark` 信息行、DU/ DUM 占位原子、坐标类型 INT/XYZ、CORRECT、CHANGE、OMIT、LOOP、IMPROPER、DONE、STOP 等控制关键字，以及原子编号、原子名、原子类型、树符号、电荷和浮点数。如图图 8 所示，说明段、控制项和原子表在显示上已有区分。

说明行、控制关键字、原子信息列和电荷数值在当前版本里都有对应匹配。对于非标准残基预处理文件，这样的分段可用于人工核对原子表和控制段。

3.2.4 Amber 力场修改文件 (FRCMOD)

FRCMOD 文件的语法定义位于 `syntaxes/frcmmod.tmLanguage.json`。该文件主要匹配 MASS、BOND、ANGLE、DIHE、IMPROPER、NONBON、CMAP 等段标识，并对不同段中的原子类型组合、浮点参数和整数参数分别着色。如图图 9 所示，各参数段之间的边界已经分开。

在该格式中，参数段标题以及各类参数行中的原子类型组合、数值项能够区分显示。其主要用途仍是人工查看新增或修改后的力场参数条目。


```
1 0 0 2
2
3 This is a remark Line
4 molecule.res
5 TYM INT 0
6 CORRECT OMIT DU BEG
7 0.0000
8 1 DUMM DU M 0 -1 -2 0.000 .0 .0 .00000
9 2 DUMM DU M 1 0 -1 1.449 .0 .0 .00000
10 3 DUMM DU M 2 1 0 1.523 111.21 .0 .00000
11 4 N N M 3 2 1 1.540 111.208 -180.000 -0.784327
12 5 H H E 4 3 2 1.011 127.816 -52.528 0.404780
13 6 CA CX M 4 3 2 1.456 19.360 -121.187 0.084200
14 7 CB CT 3 6 4 3 1.539 111.990 39.865 0.000930
15 8 CG CA S 7 6 4 1.513 116.601 53.445 -0.316457
16 9 CD1 CA B 8 7 6 1.395 120.882 -79.430 -0.043509
17 10 CE1 CA B 9 8 7 1.389 121.402 -176.679 -0.322452
18 11 CZ C B 10 9 8 1.386 119.862 -0.177 0.579275
19 12 CE2 CA B 11 10 9 1.392 120.129 -0.254 -0.322452
20 13 CD2 CA S 12 11 10 1.400 119.556 0.351 -0.043509
21 14 HD2 HA E 13 12 11 1.080 119.362 -179.517 0.072836
22 15 HE2 HA E 12 11 10 1.079 120.147 -179.957 0.104346
23 16 OH O E 11 10 9 1.383 119.350 178.170 -0.763539
24 17 HE1 HA E 10 9 8 1.077 120.511 -179.073 0.104346
25 18 HD1 HA E 9 8 7 1.074 119.028 4.519 0.072836
26 19 HB2 HC E 7 6 4 1.087 109.488 176.075 0.019836
27 20 HB3 HC E 7 6 4 1.090 107.863 -67.968 0.019836
28 21 HA H1 E 6 4 3 1.091 110.101 -80.299 0.039982
29 22 C C M 6 4 3 1.517 109.771 162.828 0.606549
30 23 O O E 22 6 4 1.229 120.422 -6.524 -0.513506
31
32
33 LOOP
34 CD2 CG
35
36 IMPROPER
37 CD2 CD1 CG CB
38 CG CE1 CD1 HD1
39 CZ CD1 CE1 HE1
```

图 8 Amber PREPIN 文件语法高亮演示 (.prepin 格式)

3.2.5 AmberTools 输入脚本 (TLEAP/相关.in 文件)

syntaxes/in.tmLanguage.json 用于 .in 文件以及 leaprc.* 模式文件。它同时照顾了 Amber MD 输入脚本、tleap 常用命令和部分 parmed 命令。代码里已经写进去的关键词包括 imin、iwrap、ntx、cut、nstlim、temp0 等 MD 参数名, 也包括 source、loadmol2、loadpdb、loadamberparams、loadAmberPrep、loadOFF、addIons、savepdb、saveamberparm、quit 等 tleap 命令。如图 10 所示, 命令、参数名和注释在脚本中能够分层显示。

在这类脚本里, 注释、参数名、命令名、常见文件格式名、离子名和变量赋值行都会分开显示。编写 Amber 输入文件和 tleap 脚本时, 命令和参数可按不同颜色层次区分。

3.2.6 Antechamber 主链文件 (MC)

MC 文件使用 syntaxes/mc.tmLanguage.json。规则识别 HEAD_NAME、TAIL_NAME、PRE_HEAD_TYPE、POST_TAIL_TYPE、CHARGE、OMIT_NAME 等字段名, 并对 NAME 行后的原子名、TYPE 行后的原子类型以及数值项进行着色。如图 11 所示, 字段名和数据值在 MC 文件中已经分开。

MC 文件里, 字段关键字、原子名、原子类型和数值项用了不同作用域。它

```
1 OL15 force field for DNA (99bsc0-betaOL1-eps-zetaOL1-chiOL4)
  see http://ffol.upol.cz
2 MASS
3 C7 12.01      0.878      sp3 aliphatic C
4 C2 12.01      0.360      sp2 C 5 memb.ring in
  purines
5 C1 12.01      0.360      sp2 C pyrimidines in
  pos. 5 & 6
6 CJ 12.01
7
8 BOND
9 C7-CT 310.0    1.526      JCC, 7, (1986), 230; AA, SUGARS
10 C7-H1 340.0    1.090      changed from 331 bsd on NMA
  nmodes; AA, RIBOSE
11 C7-OH 320.0    1.410      JCC, 7, (1986), 230; SUGARS
12 C7-OS 320.0    1.410      JCC, 7, (1986), 230; NUCLEIC ACIDS
13 C2-H5 367.0    1.080      changed from 340. bsd on C6H6
  nmodes; ADE, GUA
14 C2-N* 440.0    1.371      JCC, 7, (1986), 230; ADE, GUA
15 C2-NB 529.0    1.304      JCC, 7, (1986), 230; ADE, GUA
16 C -C1 410.0    1.444      JCC, 7, (1986), 230; THY, URA
17 CA-C1 427.0    1.433      JCC, 7, (1986), 230; CYT
18 C1-C1 549.0    1.350      JCC, 7, (1986), 230; CYT, THY, URA
19 C1-CT 317.0    1.510      JCC, 7, (1986), 230; THY
20 C1-HA 367.0    1.080      changed from 340. bsd on C6H6
  nmodes; CYT, URA
21 C1-H4 367.0    1.080      changed from 340. bsd on C6H6
  nmodes; CYT, URA
22 C1-N* 448.0    1.365      JCC, 7, (1986), 230; CYT, THY, URA
23 CJ-H1 340.0    1.090
24 CJ-CT 310.0    1.526
25 OS-CJ 320.0    1.410
26 OH-CJ 320.0    1.410
27
28 ANGLE
29 H1-C7-OH 50.0    109.50    changed based on NMA nmodes
30 H1-C7-OS 50.0    109.50    changed based on NMA nmodes
31 C7-CT-CT 40.0    109.50
32 CT-C7-CT 40.0    109.50
```

图 9 Amber FRCMOD 文件语法高亮演示 (.frcmod 格式)

```
1 source leaprc.protein.ff14SB
2 source leaprc.lipid21
3 source leaprc.water.tip3p
4 source leaprc.gaff
5 mem = loadpdb membrane.pdb
6 lig = loadmol2 N001_bcc.mol2
7 loadamberparams N001.frcmod
8 sys = combine {mem lig}
9 charge sys
10 addIonsRand sys Cl- 13
11 addIonsRand sys Na+ 13
12 saveamberparm sys system.prmtop system.inpcrd
```

图 10 tleap 输入脚本语法高亮演示 (.in 格式)

的主要作用是在查看主链定义文本时，把名称字段和参数字段先分开。

```
1 HEAD_NAME C10
2 PRE_HEAD_TYPE c3
3 TAIL_NAME C11
4 POST_TAIL_TYPE c3
5 OMIT_NAME C
6 OMIT_NAME C1
7 OMIT_NAME C2
8 OMIT_NAME C3
9 OMIT_NAME O
10 OMIT_NAME O1
11 OMIT_NAME C4
12 OMIT_NAME C5
13 OMIT_NAME N
14 OMIT_NAME C6
15 OMIT_NAME C7
16 OMIT_NAME C8
17 OMIT_NAME C9
18 OMIT_NAME O5
19 OMIT_NAME C20
20 OMIT_NAME C21
```

图 11 MC 主链文件语法高亮演示 (.mc 格式)

3.2.7 Antechamber 文件 (AC)

AC 文件的语法定义位于 `syntaxes/ac.tmLanguage.json`。这份规则识别 CHARGE、Formula 两类头部关键字；对 ATOM/HETATM 行按固定宽度拆分原子编号、原子名、残基名、残基编号、三维坐标、电荷和原子类型；对 BOND 行则区分键序号、两端原子编号、键类型和两端原子名。如图图 12 所示，AC 文件里的头部、原子段和键段能够看出不同层次。

```
34 ATOM 32 H13 LIG 1 2.215 -4.381 1.698 0.035367 hc
35 ATOM 33 H14 LIG 1 2.223 -3.051 2.867 0.035367 hc
36 ATOM 34 H15 LIG 1 4.152 -0.381 -3.544 0.084700 h1
37 ATOM 35 H16 LIG 1 5.419 -1.405 -2.853 0.084700 h1
38 ATOM 36 H17 LIG 1 6.778 0.508 -2.255 0.075200 hx
39 ATOM 37 H18 LIG 1 5.479 1.660 -2.541 0.075200 hx
40 ATOM 38 H19 LIG 1 4.694 0.415 -5.364 0.069200 hx
41 ATOM 39 H20 LIG 1 5.812 1.421 -6.253 0.069200 hx
42 ATOM 40 H21 LIG 1 3.964 2.408 -3.994 0.053700 hc
43 ATOM 41 H22 LIG 1 3.744 2.689 -5.719 0.053700 hc
44 ATOM 42 H23 LIG 1 5.108 3.430 -4.876 0.053700 hc
45 ATOM 43 H24 LIG 1 8.270 1.660 -3.545 0.069200 hx
46 ATOM 44 H25 LIG 1 7.034 2.921 -3.761 0.069200 hx
47 ATOM 45 H26 LIG 1 8.396 1.505 -6.134 0.053700 hc
48 ATOM 46 H27 LIG 1 9.041 2.986 -5.403 0.053700 hc
49 ATOM 47 H28 LIG 1 7.452 3.003 -6.168 0.053700 hc
50 BOND 1 1 2 1 C C1
51 BOND 2 1 19 1 C H
52 BOND 3 1 20 1 C H1
53 BOND 4 1 21 1 C H2
54 BOND 5 2 3 1 C1 C2
55 BOND 6 2 5 1 C1 C4
56 BOND 7 2 8 1 C1 C7
57 BOND 8 3 4 1 C2 C3
58 BOND 9 3 22 1 C2 H3
```

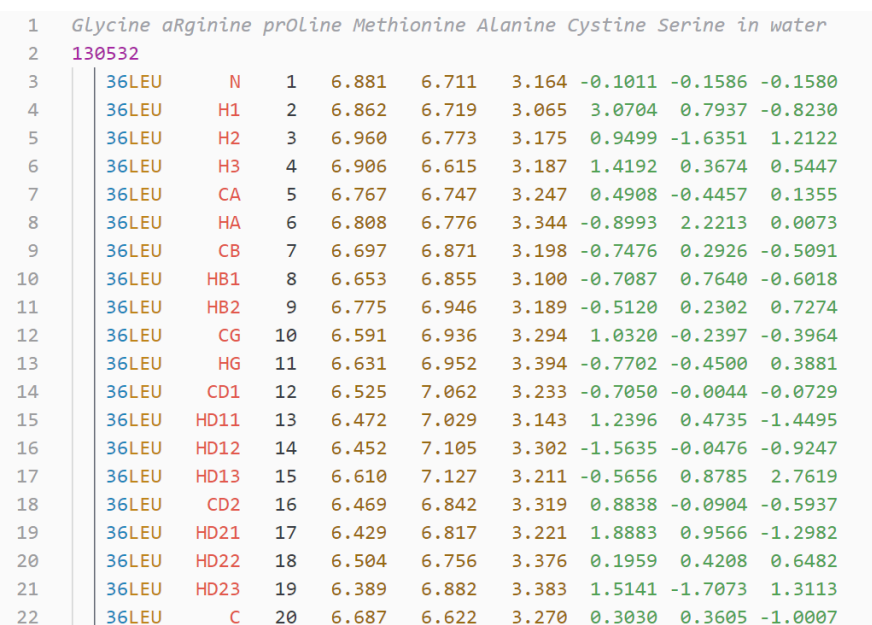
图 12 AC 文件语法高亮演示 (.ac 格式)

头部说明、原子记录和键记录在这里分别用了独立的匹配范围。对 **An-techamber** 中间文件来说，可据此区分原子信息和键信息。

3.3 Gromacs 文件格式

3.3.1 Gromacs 坐标文件（GRO）

GRO 文件的高亮规则定义在 `syntaxes/gro.tmLanguage.json`。相关规则按列宽处理：时间步字符串、标题行、总原子数行、盒子向量行先行区分，正文再按固定列宽区分残基编号、残基名、原子名、坐标列和速度列。代码中预留了原子编号字段的位置，但该项当前的`match`为空，因此现阶段重点仍是名称列和数值列。如图图 13所示，GRO 文件的列结构在编辑器里可以看出。



1	Glycine aRginine prOline Methionine Alanine Cystine Serine in water								
2	130532								
3	36LEU	N	1	6.881	6.711	3.164	-0.1011	-0.1586	-0.1580
4	36LEU	H1	2	6.862	6.719	3.065	3.0704	0.7937	-0.8230
5	36LEU	H2	3	6.960	6.773	3.175	0.9499	-1.6351	1.2122
6	36LEU	H3	4	6.906	6.615	3.187	1.4192	0.3674	0.5447
7	36LEU	CA	5	6.767	6.747	3.247	0.4908	-0.4457	0.1355
8	36LEU	HA	6	6.808	6.776	3.344	-0.8993	2.2213	0.0073
9	36LEU	CB	7	6.697	6.871	3.198	-0.7476	0.2926	-0.5091
10	36LEU	HB1	8	6.653	6.855	3.100	-0.7087	0.7640	-0.6018
11	36LEU	HB2	9	6.775	6.946	3.189	-0.5120	0.2302	0.7274
12	36LEU	CG	10	6.591	6.936	3.294	1.0320	-0.2397	-0.3964
13	36LEU	HG	11	6.631	6.952	3.394	-0.7702	-0.4500	0.3881
14	36LEU	CD1	12	6.525	7.062	3.233	-0.7050	-0.0044	-0.0729
15	36LEU	HD11	13	6.472	7.029	3.143	1.2396	0.4735	-1.4495
16	36LEU	HD12	14	6.452	7.105	3.302	-1.5635	-0.0476	-0.9247
17	36LEU	HD13	15	6.610	7.127	3.211	-0.5656	0.8785	2.7619
18	36LEU	CD2	16	6.469	6.842	3.319	0.8838	-0.0904	-0.5937
19	36LEU	HD21	17	6.429	6.817	3.221	1.8883	0.9566	-1.2982
20	36LEU	HD22	18	6.504	6.756	3.376	0.1959	0.4208	0.6482
21	36LEU	HD23	19	6.389	6.882	3.383	1.5141	-1.7073	1.3113
22	36LEU	C	20	6.687	6.622	3.270	0.3030	0.3605	-1.0007

图 13 Gromacs GRO 文件语法高亮演示（.gro 格式）

标题、总数、盒向量、残基名、原子名、坐标和速度列在显示上是分开的。GRO 文件本来就依赖列结构，这类高亮主要也是围绕列结构展开。

3.3.2 Gromacs 原子类型文件（ATP）

ATP 文件使用 `syntaxes/atp.tmLanguage.json`。该规则内容较短，主要处理分号注释、浮点数、一般原子类型字符串以及带 **amber** 前缀的类型名。如图图 14所示，ATP 文件里类型名和数值列已经区分开。

ATP 这部分没有很复杂的层级，主要就是把注释、类型名和参数数值分开。用于查看原子类型表 and 对应参数时，可以先看名称列，再看数值列。

```
1 ; This force field generated by charmm2gmx.py from
2 ; multiple charmm parameter files
3 ; and multiple charmm topology files
4   ALG1      26.98154 ; Aluminum, for ALF4, ALF4-
5   | BAR      137.32700 ; Barium Ion
6   BRGA1     79.90400 ; BRET, bromoethane
7   BRGA2     79.90400 ; DBRE, 1,1-dibromoethane
8   BRGA3     79.90400 ; TBRE, 1,1,1-dibromoethane
9   BRGR1     79.90400 ; BROB, bromobenzene
10  | C        12.01100 ; carbonyl C, peptide backbone
11  | CA       12.01100 ; aromatic C
12  CAD       112.41100 ; cadmium (II) cation
13  CAI       12.01100 ; aromatic C next to CPT in trp
14  CAL       40.08000 ; Calcium Ion
15  | CC       12.01100 ; carbonyl C, asn,asp,gln,glu,cter,ct2
16  CC1A      12.01100 ; alkene conjugation
17  CC1B      12.01100 ; alkene conjugation
18  | CC2      12.01100 ; alkene conjugation
19  CC201     12.01100 ; sp2 carbon in amides, aldoses
20  CC202     12.01100 ; sp2 carbon in carboxylates
```

图 14 Gromacs ATP 原子类型文件语法高亮演示 (.atp 格式)

3.3.3 Gromacs 原子重命名文件 (ARN)

ARN 文件的语法定义位于 `syntaxes/arn.tmLanguage.json`。它识别分号注释、浮点数、行首的残基名、其后的原子名以及后续原子类型字符串。这个文件属于映射类文本，实现重点就是把名称列和数值列分开。

ARN 文件中，注释、残基名、原子名和原子类型字符串是分别匹配的。这种写法可用于人工检查重命名规则文本里的字段顺序。

3.3.4 Gromacs 残基类型文件 (RTP)

RTP 文件使用 `syntaxes/rtp.tmLanguage.json`。规则里识别分号注释、`torsion`和`un`等特殊说明行、方括号分段名、原子名、四元原子组、原子类型、电荷以及一般数值项。像 `[ atoms ]`、`[ bonds ]`、`[ impropers ]` 这些分段内常见字段，当前都做了区分。

RTP 里能够看到较清楚的层次：分段名称、原子组、原子类型、电荷和数值参数是分开的。编辑或阅读 RTP 文本时，可以先定位分段，再查看字段列。

3.3.5 Gromacs 氢数据库文件 (HDB)

HDB 文件的语法定义位于 `syntaxes/hdb.tmLanguage.json`。规则识别分号注释、行首残基名、数量字段、原子名组以及数值项。HDB 的文本组织比较规整，所以这里主要还是靠列位置和行结构做匹配。

HDB 里，残基名、计数字段、原子名组合和数值列是分别显示的。对这类加氢规则文件来说，可据此查看条目结构。

3.3.6 Gromacs 残基到珠映射文件 (R2B)

R2B 文件使用 `syntaxes/r2b.tmLanguage.json`。实现较简单，主要对分号注释、字符串形式的残基名以及数值项进行区分。

R2B 的高亮规则相对简洁，注释、名称字段和数值字段分别显示，更接近轻量级映射表的展示方式。

3.3.7 Gromacs 终端数据库文件 (TDB)

TDB 文件的语法定义位于 `syntaxes/tdb.tmLanguage.json`。这一项支持分号注释、方括号分段名、原子名、一组四到五个原子名的规则行、原子类型、原子质量、电荷以及一般数值项。这里的重点还是把终端修饰规则里的名称字段和参数字段拆开。如图 15 所示，其中 (a) 为 MTP 文件显示效果，(b) 为 TDB 文件显示效果；两类文件都能把分段和参数列分开。



图 15 Gromacs MTP 突变拓扑文件和 TDB 终端数据库文件语法高亮演示

在 TDB 里，分段、原子名组、类型、质量和电荷用了不同作用域。浏览终端数据库文件时，规则段落的边界会更清楚一些。

3.3.8 Gromacs 突变拓扑文件 (MTP)

MTP 文件使用 `syntaxes/mtp.tmLanguage.json`。规则识别分号注释、以 `default` 开头的关键字、方括号分段名、原子名、原子类型、电荷和一般数值项。代码里对部分旋转相关行和四元原子组还写了专门模式。

MTP 当前能把默认关键字、分段、原子名、电荷和数值列分出层次。阅读突变拓扑文本时，可以先定位状态块，再查看原子字段。

3.3.9 Gromacs 虚拟位点定义文件（VSD）

VSD 文件的语法定义位于 `syntaxes/vsd.tmLanguage.json`。这是 0.0.7 版本新增支持的文件类型。规则识别分号注释、关键字 `planar`、方括号分段、原子名、原子类型和数值项，范围不大，但和当前代码是一致的。

VSD 这部分的范围不大，关键字、分段、原子名和参数值是分别显示的。用于查看虚拟位点定义文本中的名称列和参数列已经够用。

3.4 小分子文件格式

3.4.1 MOL2 分子结构文件（MOL2）

MOL2 文件使用 `syntaxes/mol2.tmLanguage.json`。这份语法和前面几类文件不太一样，实际是通过 `repository.structure` 中的规则来工作。当前可识别注释行、以 `@<TRIPOS>` 形式出现的结构段标识、原子编号、原子名、原子类型、残基编号、残基名、电荷和键类型。因为它用了 `include` 引用内部仓库规则，所以文件结构看起来也和其他语法文件不完全一样。如图 16 所示，其中 (a) 为 MOL2 文件显示效果，(b) 为 SDF 文件显示效果。

MOL2 里，结构段标识、原子名、原子类型、残基信息、电荷和键类型是分开显示的。这个格式可用于阅读小分子结构文本里的原子表和键表。

3.4.2 结构数据文件（SDF）

SDF 文件的语法定义位于 `syntaxes/sdf.tmLanguage.json`。当前版本里，它会识别单独的字符串行、`V2000` 标志、`M` `END` 和 `$$$$` 结束标记、属性头 `> <...>`、三维坐标列、元素符号、键连接行和键级。

SDF 里，文件头标记、坐标列、元素符号、键连信息、属性头和结束标识是分开显示的。查看这类文件时，文本块边界和原子坐标区会更清楚。

3.5 增强采样文件格式

3.5.1 PLUMED 输入文件

PLUMED 文件使用 `syntaxes/plumed.tmLanguage.json`。当前规则识别注释行、动作关键字 `GROUP`、`MOLINFO`、`PRINT`、`ANGLE`、`TORSION`、`DISTANCE`、`COM`、`GYRATION`、`VOLUME`，偏置关键字 `LOWER_WALLS` 和 `UPPER_WALLS`，通用参数名 `ATOMS`、`ARG`、`FILE`、`STRIDE`、`NDX_FILE`、`NDX_GROUP`、`RESTRAINT`、



图 16 MOL2 分子结构文件和 SDF 结构数据文件语法高亮演示

METAD，以及参数值和数值项。如图图 17所示，PLUMED 输入文件里的动作关键字和参数项已经能分层显示。

```

1  # Define the groups
2  ligc2: GROUP ATOMS=16069
3  popc: GROUP NDX_FILE=index.ndx NDX_GROUP=MEMB
4  com: COM ATOMS=popc
5  dist: DISTANCE ATOMS=ligc2,com
6  # Add a Lower wall at (2.3 nm)
7  restraint: LOWER_WALLS ARG=dist AT=2.3 KAPPA=150.0 EXP=2 EPS=1 OFFSET=0
8  PRINT ARG=restraint.bias FILE=bias

```

图 17 PLUMED 输入文件语法高亮演示 (.plumed.dat 格式)

PLUMED 的显示层次已经分开，动作关键字、偏置关键字、参数名、变量名和数值项使用不同作用域。编写或阅读输入脚本时，可据此区分动作段和参数段。

3.6 实际用途

3.6.1 文本阅读和人工检查

本软件的直接用途在于文本阅读和编辑辅助。对于列宽固定或字段密集的文件，语法高亮可以帮助用户区分不同字段所在位置。例如：

- PDB、GRO、AC 这类固定列宽文件中，坐标列和名称列可以分开查看
- PRMTOP、LIB、TDB、RTP 这类分块或分段文件中，可以先定位分段标

识

- **tleap**、**PLUMED** 等脚本文件中，命令和参数名可以按不同颜色层次查看

这里的“检查”指人工阅读时的辅助，不等同于程序自动校验，也不表示软件能够给出结构正确性结论。

3.6.2 跨格式编辑场景

分子模拟工作流通常会同时涉及多种文本格式。**md-highlighter** 的作用不是串联这些格式之间的数据转换，而是在同一个编辑器中为不同文件提供较一致的显示方式。当前版本可用于以下编辑场景：

- 在同一 VSCode 会话中切换查看 PDB、PSF、PRM 等 NAMD/CHARMM 相关文件
- 并列编辑 MOL2、PREPIN、FRCMOD、PRMTOP 等 Amber/AmberTools 文件
- 查看 GRO、RTP、TDB、MTP、VSD 等 Gromacs 配置文本
- 编写 `.plumed.dat` 文件时区分动作块和参数块

四、 未来发展

4.1 计划改进

从 README 和现有语法文件看，后续仍有一些可以继续完善的方向：

- 继续补充更多分子模拟相关文本格式，如 LAMMPS 软件等
- 调整部分规则对特殊原子命名方式的兼容性
- 针对已有语法文件中的已知注释和 TODO 继续细化匹配范围
- 结合实际使用情况修正少数过宽或过窄的正则规则

4.2 总结

md-highlighter 当前版本的实现重心，是为分子动力学相关文本文件提供编辑器级语法高亮。源码中已注册 23 个语言 ID，并配套 23 个语法定义文件。当前版本完成的是文件识别、文本匹配和作用域分配，不包含独立计算引擎、网络服务或数据库模块。